SAFESTEER AI-DRIVER MONITORING SYSTEM

MINI PROJECT REPORT

Submitted by

ATHIRA S

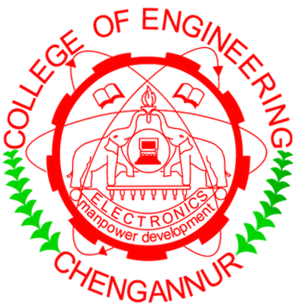
CHN25MCA2020

to

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of Degree in

Master of Computer Applications



DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR, ALAPPUZHA

OCTOBER 2026

DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR

ALAPPUZHA

CERTIFICATE

This is to certify that the project report titled ***SAFESTEER AI-DRIVER MONITORING SYSTEM*** *is a bonafide record of the* ***20MCA245 MINI PROJECT*** *presented by* ***ATHIRA S*** *(CHN25MCA2020), Third Semester Master of Computer Applications student, under our guidance and supervision. This project is submitted in partial fulfillment of the requirements for the award of the degree* ***Master of Computer Applications*** *of* ***APJ Abdul Kalam Technological University****.*

Smt. Arya Jayakumar
Project Guide
Assistant Professor
Dept. of Computer Engineering

Smt. Josmi Jose
Project Coordinator
Assistant Professor
Dept. of Computer Engineering

Dr. Renu George

Professor

Head of the Department

Dept. of Computer Engineering

DECLARATION

I undersigned hereby declare that the project report "**SAFESTEER AI-DRIVER MONITORING SYSTEM**" , submitted for partial fulfillment of the requirements for the award of degree of Master of Computer Application of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Smt. Arya Jayakumar**, Assistant Professor, Department of Computer Engineering. This submission represents my ideas in my own words, and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in this submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Chengannur **ATHIRA S**

Date : **CHN25MCA2020**

ACKNOWLEDGEMENT

This work would not have been possible without the support of many people. First and foremost, I give thanks to Almighty God who gave me the inner strength, resources, and ability to complete my project successfully.

I would like to thank **Dr. Hari V.S** , The Principal , College of Engineering Chengannur, for providing the best facilities and atmosphere for the project completion and presentation. Special thanks to **Dr. Renu George** , HOD , Department of Computer Engineering, for her exceptional support, guidance, and encouragement throughout the project. I would also like to thank my project coordinators **Smt. Josmi Jose**, Assistant Professor, Department of Computer Engineering , **Smt. Santhy Viswam** , Assistant Professor, Department of Computer Engineering ,**Smt. Celin T**,Assistant Professor, Department of Computer Engineering , for their extended help and support during the project. I would also like to thank **Smt. Arya Jayakumar** , Assistant Professor, Department of Computer Engineering, who also took on the role of my project guide , for her extended help and support during the project.

I would like to thank my dear friends and faculty for extending their cooperation and encouragement throughout the project work, without which I would never have completed the project this well. Thank you all for your love and also for being very understanding.

ATHIRA S

CHN25MCA2020

ABSTRACT

”SafeSteer AI-Driver Monitoring System” is a web-based platform developed to help fleet managers monitor driver alertness and prevent road accidents caused by driver drowsiness and distraction. Conventional fleet management depends mostly on manual supervision and on checking records after an accident has already happened, and such methods cannot stop sudden crashes caused by driver sleepiness. The proposed system addresses this problem by tracking driver focus live through a camera and Computer Vision. It captures live video of the driver, identifies eye-closure patterns from face-tracking data, and considers factors such as the eye aspect ratio and the time duration of eye closure to detect drowsiness and distraction instantly. When a risk is detected, the system warns the driver through multi-stage audio alerts consisting of voice warnings, beeps, and horn sounds, and it provides an emergency call simulation. Every event is saved as an incident log in the database, and the fleet manager can view live alerts, incident logs, driver history, risk levels, and safety trends through an admin dashboard. The system is developed using Python and Flask for the backend, OpenCV and MediaPipe for face tracking and drowsiness detection, SQLite for data storage, PyGame and Text-to-Speech for audio processing, and HTML, CSS, and JavaScript for the web interface. The system aims to provide better fleet control, tamper-proof incident records, and organizational safety decisions through a simple, practical, and secure monitoring platform.

CONTENTS

Declaration i

Acknowledgement ii

Abstract iii

List of Figures vii

List of Tables viii

1 INTRODUCTION 1

1.1 Project Area 1

1.2 Objectives 1

2 LITERATURE REVIEW 3

2.1 Dynamic Driver Drowsiness Detection with Attention-Enhanced CNNs for Real-Time Monitoring and Road Safety Applications 3

2.2 Real-Time Driver Drowsiness Detection Using Transformer Architectures: A Novel Deep Learning Approach 3

2.3 Driver Drowsiness Shield (DDSH): A Real-Time Driver Drowsiness Detection System 4

2.4 Drowsiness Detection in Drivers: A Systematic Review of Deep Learning-Based Models 4

3 PROBLEM DEFINITION AND MOTIVATIONS 5

3.1 Existing System 5

3.2 Limitations 5

3.3 Problem Statement 6

3.4 Proposed System 6

4 REQUIREMENT ANALYSIS 8

4.1 Hardware Requirements 8

4.2 Software Requirements 8

4.3 System Features 9

4.4 Non-Functional Requirements 10
4.4.1 Performance Requirements 10
4.4.2 Quality Requirements 10

5 DESIGN AND IMPLEMENTATION 11
5.1 Overall Design 11
5.2 System Design 11
5.2.1 Client-Side Design 12
5.2.2 Server-Side Design 12
5.3 System Architecture 13
5.4 Use Case Diagram 14
5.5 Data Flow Diagrams (DFD) 15
5.6 Database Design 17
5.7 Module Description 17
5.7.1 Management and Reporting Module 18
5.7.2 Monitoring and Detection Module 18

6 REPORT OF PROJECT IMPLEMENTATION 19
6.1 Implementation Plan 19
6.1.1 Setup and Environment Preparation 19
6.1.2 Backend Implementation 19
6.1.3 Frontend Implementation 20
6.1.4 Security and Data Handling 20
6.1.5 Project Schedule 21
6.2 Testing and Various Types of Testing Used 21
6.2.1 Unit Testing 21
6.2.2 Integration Testing 23
6.2.3 System Testing 24

7 RESULTS AND DISCUSSION 26
7.1 Advantages 26
7.2 Limitations 27
7.3 Screenshots 28

8 CONCLUSION AND FUTURE SCOPE 32

8.1 Conclusion 32
8.2 Future Scope 32

REFERENCES **33**

LIST OF FIGURES

5.1 System Architecture 14
5.2 Use Case Diagram for SafeSteer AI-Driver Monitoring System 15
5.3 DFD Level 0 16
5.4 DFD Level 1 - Admin 16
5.5 DFD Level 1 - Driver 17

7.1 Home Page 28
7.2 Platform Features 28
7.3 Driver Registration 28
7.4 Sign In 29
7.5 Driver Dashboard 29
7.6 Trip Incident History 29
7.7 Driver Dashboard with Active AI Monitoring 30
7.8 Trip Incident History with Detected Incidents 30
7.9 Admin Fleet Management Console 30
7.10 Admin All Fleet Incidents 31

LIST OF TABLES

6.1 Unit Test Cases and Results 22
6.2 Integration Test Cases and Results 24
6.3 System Test Cases and Results 25

CHAPTER 1

INTRODUCTION

1.1 Project Area

Fleet management needs constant driver checks to ensure safe trips on the road. Drowsy driving is one of the serious causes of road accidents, because a driver who is sleepy reacts slowly, loses focus on the road, and may fall asleep for a few seconds without any warning. For commercial fleets, such events can lead to loss of life, damage to vehicles and goods, and loss to the organization.

Conventionally, fleet tracking relies mostly on manual supervision and on checking records after an accident. Basic vehicle tracking and GPS logs show where a vehicle is, but they do not show the state of the driver. These older methods cannot stop sudden crashes caused by driver sleepiness, and fleet managers get no live information about whether a driver is losing focus while the vehicle is on the road.

" **SafeSteer AI-Driver Monitoring System** " is proposed as a web platform that tracks driver focus live to spot drowsiness right away. It captures live video feeds of the driver, uses face tracking to identify eye-closure patterns, and uses Computer Vision to detect drowsiness and distraction. The system warns the driver through voice warnings, beeps, and horn sounds, provides an emergency call simulation, and stores tamper-proof incident logs that fleet managers can review. The platform supports two types of users, namely the driver (employee) and the fleet manager (admin), each with a separate set of permitted operations.

Developed using Python, Flask, OpenCV, MediaPipe, SQLite, and web technologies, the system combines web application development, Computer Vision, audio processing, and database management in a practical system for fleet safety. It gives fleet managers complete control over driver monitoring through a dashboard of live alerts, incident logs, driver history, and reports.

1.2 Objectives

1. **To develop an integrated platform for driver safety monitoring:** The system aims to provide a single web platform where drivers and fleet managers can carry out their activities related to driver monitoring, incident review, and fleet safety. The platform reduces the dependence on manual supervision and on checking records after an accident.

2. **To detect driver drowsiness and distraction in real time:** The system captures live video feeds of drivers, identifies eye-closure patterns from face-tracking data, and uses Computer Vision to detect drowsiness and distraction instantly, considering factors such as the eye aspect ratio and the time duration.
3. **To warn drivers through instant audio alerts:** The system triggers immediate multi-stage audio alerts consisting of voice warnings, beeps, and horn sounds, together with emergency call warnings, so that a drowsy driver is alerted before an accident happens.
4. **To maintain tamper-proof incident logs:** The system captures the distraction type, time, and system alerts of every incident and stores them as tamper-proof logs in the database, so that they are available for fleet review and safety audits.
5. **To manage drivers and vehicles:** The system enables the fleet manager to add, update, view, search, and delete driver profiles and to manage vehicle assignments and company login credentials.
6. **To provide operational insight through reports and analytics:** The system displays driver history, risky habits, incident trends, and risk levels so that fleet managers can review driver performance, enforce safety standards, and reduce accidents.
7. **To provide secure access to the platform:** The system provides registration and login with separate permissions for drivers and the fleet manager, so that each user can access only the operations permitted for the role.

CHAPTER 2

LITERATURE REVIEW

2.1 Dynamic Driver Drowsiness Detection with Attention-Enhanced CNNs for Real-Time Monitoring and Road Safety Applications

This paper, published in 2026, proposes a CNN with an attention mechanism for dynamic driver drowsiness detection. The study used pre-trained models such as VGG16, VGG19, ResNet, and DenseNet along with a custom attention-based CNN, and facial landmarks, eye closure, and yawning features were extracted from the NTHU-DDD dataset. The proposed CNN with attention achieved 99.63% accuracy with near-perfect precision, recall, and F1-score, and the attention mechanism improved the focus on the eye and mouth regions. This paper is relevant to the proposed system because it uses facial landmarks and eye closure as indicators, which is the basis of the face-mapping approach in the proposed system. However, existing systems still struggle with lighting variation, occlusion, and dependence on single indicators, and real-world deployment still requires optimization and multimodal integration. [1]

2.2 Real-Time Driver Drowsiness Detection Using Transformer Architectures: A Novel Deep Learning Approach

This paper, published in 2025, studies real-time driver drowsiness detection using transformer-based deep learning. The study used Vision Transformer (ViT), Swin Transformer, VGG19, DenseNet169, ResNet50V2, and MobileNet, with eye images from the MRL Eye Dataset that were preprocessed using normalization, augmentation, and Haar Cascade eye detection, and an alarm is generated in real time when the eyes remain closed. The Vision Transformer achieved 99.15% accuracy and the Swin Transformer achieved 99.03% accuracy, and the models showed strong performance under different lighting conditions. This paper is relevant to SafeSteer AI-Driver Monitoring System because it also treats prolonged eye closure as the main sign of drowsiness and raises a real-time alarm. However, the models require large datasets and high computational resources, and performance can still be affected by occlusion, sunglasses, and real-world deployment challenges, whereas the pro-

posed system concentrates on a practical fleet platform built on face tracking, eye aspect ratio, alerts, and incident logging. [2]

2.3 Driver Drowsiness Shield (DDSH): A Real-Time Driver Drowsiness Detection System

This paper, published in 2025, presents a real-time driver drowsiness detection system that uses transfer learning with the MobileNet CNN. The MRL Eye Dataset was used to classify eye states as open or closed, and the classifier was integrated with real-time camera monitoring and an alarm system. The system achieved 90% accuracy, 100% precision, 83.3% recall, and a 90.9% F1-score, and it detected drowsiness successfully in real time. This paper is relevant to the proposed system because it shows that a camera-based eye-state classifier combined with an alarm can detect drowsiness while the driver is driving. However, its accuracy is lower than recent transformer-based models and its performance may vary with environmental conditions, driver behavior differences, glasses, and occlusions. It also does not provide a fleet manager dashboard with incident history. [3]

2.4 Drowsiness Detection in Drivers: A Systematic Review of Deep Learning-Based Models

This paper, published in 2025, is a systematic review that follows the PRISMA 2020 guidelines and reviews 81 studies published from 2015 to 2025 that use CNN, RNN, LSTM, Transformer, and hybrid models. It found that most deep learning models achieved a median accuracy and F1-score above 0.95 under simulated and real-world conditions. This paper is relevant to the proposed system because it confirms that deep learning and Computer Vision are widely used for driver drowsiness detection and identifies the open problems of the field. However, it reports a lack of demographic diversity, inconsistent evaluation metrics, limited dataset transparency, privacy concerns, and weak real-world validation, which shows the need for a simple, practical, and secure system that can be used in a real organization. [4]

CHAPTER 3

PROBLEM DEFINITION AND MOTIVATIONS

3.1 Existing System

Existing fleet management mostly relies on manual supervision and on checking records after an accident, and basic vehicle tracking and GPS logs show only the position of the vehicle. Fleet managers do not get a live view of the state of the driver, so a driver who is becoming sleepy cannot be warned in time. Existing research addresses driver drowsiness mainly through deep learning models for eye-state or fatigue classification, and most of it focuses on detection accuracy rather than on a complete workflow for drivers and fleet managers. It makes limited use of Computer Vision for live drowsiness and distraction detection in commercial fleet drivers, and it gives less attention to combining instant multi-stage audio alerts, emergency simulation, and tamper-proof log history. The proposed SafeSteer AI-Driver Monitoring System addresses these limitations by providing a single web platform that combines live drowsiness detection, multi-stage audio alerts, an emergency call simulation, tamper-proof incident logging, and a fleet manager dashboard.

3.2 Limitations

- **Manual Supervision:** Traditional fleet monitoring depends on manual supervision and cannot check every driver at all times.
- **Post-Accident Checking:** Records are checked only after an accident has happened, so the accident cannot be prevented.
- **Driver Sleepiness Not Detected:** Older methods cannot stop sudden crashes caused by driver sleepiness.
- **Basic Tracking Only:** Most existing systems focus on simple vehicle tracking or basic GPS logs rather than real-time driver state monitoring.
- **Limited Use of Computer Vision:** Computer Vision is used only to a limited extent for detecting live drowsiness and distraction in commercial fleet drivers.
- **No Instant Driver Warning:** Drivers do not receive instant voice, beep, or horn alerts when they become drowsy.

- **Weak Incident Records:** Incident records are not always stored in a tamper-proof form that can be used for safety audits.
- **Isolated Research Solutions:** Existing work concentrates on detection models and does not combine multi-stage audio alerts, emergency simulation, and log history.
- **Lack of Integrated Operational Platform:** Existing approaches do not provide a simple and practical workflow connecting drivers and the fleet manager.
- **Limited Operational Reporting:** Fleet managers lack dashboards for driver history, incident trends, and risk levels.

3.3 Problem Statement

To build an AI-based system that detects driver drowsiness in real time using live video feeds and visual tracking, helping fleet managers prevent accidents by sending instant audio alerts and saving tamper-proof incident logs. The system aims to replace manual supervision and post-accident record checking with live monitoring, to warn drivers immediately when drowsiness or distraction is detected, and to provide fleet managers with an efficient way to review incidents and driver safety.

3.4 Proposed System

The SafeSteer AI-Driver Monitoring System is a single web platform for drivers (employees) and the fleet manager (admin). It captures live video feeds of drivers during vehicle operation, identifies eye-closure patterns from face-tracking data, and uses Computer Vision models to detect drowsiness and distraction instantly, considering key factors such as the eye aspect ratio and the time duration. When drowsiness is detected, it triggers immediate audio alerts, horn sounds, and emergency call warnings to the driver, and it sends incident logs to the fleet manager dashboard to enforce safety and prevent accidents. The system is built using Flask with an SQLite database, and it uses OpenCV and MediaPipe for drowsiness detection, PyGame and Text-to-Speech for audio processing, and HTML, CSS, and JavaScript for the web interface. Overall, the system aims to provide a simple, practical, and secure organizational AI system that helps fleet managers monitor driver safety efficiently. The key features of the proposed system include:

1. **User Registration and Login :** Allows drivers to register and securely log in, and allows the fleet manager to log in as admin, with separate permissions for each role.
2. **Driver and Vehicle Management :** Allows the admin to add, update, delete, and search driver records and to manage vehicle assignments.

3. **Live Video Monitoring :** Captures and analyzes live video feeds of drivers during vehicle operation.
4. **Drowsiness and Distraction Detection :** Identifies eye-closure patterns from face-tracking data and detects drowsiness and distraction using the eye aspect ratio and the time duration.
5. **Multi-Stage Audio Alerts :** Provides instant voice warnings, beeps, and horn sounds to the driver when drowsiness is detected.
6. **Emergency Call Simulation :** Simulates an emergency rescue call when a critical incident is detected.
7. **Tamper-Proof Incident Logging :** Records the distraction type, time, severity, and system alerts of each incident in the database.
8. **Driver Dashboard and History :** Allows drivers to view their own profile, vehicle details, and incident history in read-only form.
9. **Administration and Analytics :** Allows the admin to manage drivers, vehicles, and alerts, and provides driver history, incident trends, and risk levels.
10. **Live Alerts and Reports :** Displays live safety alerts and incident summaries directly on the admin dashboard.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Hardware Requirements

- **Computer/Laptop:** A computer or laptop with a minimum of 4 GB RAM and a dual-core processor
- **Storage:** At least 10 GB of free storage space for the source code, database, and tools
- **Camera:** A web camera for capturing live driver video feeds
- **Display:** A monitor or screen for viewing the management dashboard
- **Optional Device:** A smartphone or tablet for accessing the system through a web browser

4.2 Software Requirements

- **Operating System:** Windows, Linux, or macOS
- **Programming Languages:**
  - **Python:** Used for backend development and Computer Vision
  - **HTML, CSS, JavaScript:** Used for frontend development
- **Framework:**
  - **Flask (Python):** Used to build the backend web application and handle requests, routing, and the connection between the dashboard, the database, and the vision models
- **Database:**
  - **SQLite:** A relational database used to store drivers, vehicles, distraction logs, admin details, and safety reports.
- **Libraries:**
  - **OpenCV and MediaPipe:** Used for capturing video and for face tracking and drowsiness detection.
  - **PyGame and Text-to-Speech:** Used for audio processing, including voice warnings, beeps, and horn sounds.

- **Development Tools:**
  - **Visual Studio Code:** For coding and development
  - **Web Browser (Chrome, Edge, or Firefox):** For running and testing the application

4.3 System Features

1. **User Registration:** Allows drivers to create accounts by providing their name, email, phone number, vehicle number, and password.
2. **User Login:** Authenticates drivers and the fleet manager with a secure login and gives access to the features permitted for the role.
3. **Driver Management:** Allows the admin to add, update, delete, and search driver records and to manage vehicle assignments.
4. **Real-Time Monitoring:** Analyzes live driver video feeds using the camera and detects eye closures and signs of fatigue.
5. **Drowsiness Detection:** Detects driver fatigue and eye closures using Computer Vision with OpenCV and MediaPipe.
6. **Audio Alerts:** Provides instant voice warnings, beeps, and horn sounds, and emergency call simulations.
7. **Incident Logging:** Records distraction events with the date, time, and severity in tamper-proof logs.
8. **Fleet Management:** Allows the admin to monitor vehicle assignments and track risky driver behaviors.
9. **Driver Dashboard:** Allows drivers to view their profile, vehicle details, and their own incident history in read-only form.
10. **Reports and Analytics:** Displays driver history, incident trends, and safety performance.
11. **Logout:** Allows users to securely end their sessions.
12. **Responsive Web Interface:** Provides a simple interface that adapts to mobile phone, tablet, and desktop browser sizes.

4.4 Non-Functional Requirements

4.4.1 Performance Requirements

1. **Fast Detection:** The system should process camera feeds and detect drowsiness quickly.
2. **Instant Alerts:** Audio alerts should be triggered without unnecessary delay when drowsiness is detected.
3. **Efficient Database Access:** The system should retrieve and store data efficiently in the relational database.
4. **Report Generation:** Driver history and incident reports should be generated within a reasonable time.
5. **Smooth Interaction:** The interface should allow smooth navigation between the driver dashboard, incident logs, and admin dashboard pages.

4.4.2 Quality Requirements

1. **Usability:** The application should be simple and easy for the fleet manager to use.
2. **Reliability:** The system should provide consistent and accurate alert warnings.
3. **Security:** Driver data and incident reports should be securely stored and accessed.
4. **Maintainability:** Separate modules for driver management, monitoring, detection, incident logging, and reports should make the system easy to maintain.
5. **Scalability:** The system should support adding more drivers and vehicle logs in the future.
6. **Responsiveness:** The interface should work properly on mobile phones, tablets, and desktop computers.
7. **Compatibility:** The application should work properly on commonly used modern web browsers.
8. **Data Integrity:** The system should maintain accurate and consistent driver, vehicle, and incident data in the database.

CHAPTER 5

DESIGN AND IMPLEMENTATION

The proposed system is a web-based application named SafeSteer AI-Driver Monitoring System, designed to help fleet managers monitor driver safety and prevent accidents caused by drowsiness. The system provides a centralized platform where drivers can register and view their own details and incident history, the Computer Vision engine monitors the live video feed of the driver, and the fleet manager manages drivers and vehicles and reviews live alerts, incident logs, and reports. The application runs on a Flask backend with an SQLite database and uses OpenCV and MediaPipe for drowsiness detection.

5.1 Overall Design

The SafeSteer AI-Driver Monitoring System is a web-based system designed to combine live driver monitoring, drowsiness detection, multi-stage audio alerts, emergency call simulation, and tamper-proof incident logging. It uses HTML, CSS, and JavaScript for the user interface, Flask for server-side processing, SQLite for data storage, and OpenCV and MediaPipe for the Computer Vision engine, together with PyGame and Text-to-Speech for audio alerts. The system allows drivers to register and view their own records, allows the Computer Vision engine to analyze the live video feed and detect drowsiness, and allows the fleet manager to manage drivers and vehicles, view live alerts, and review incident logs and reports. Overall, the system aims to provide a simple and organized platform for driver safety monitoring in a fleet.

5.2 System Design

The system is structured into several layers: user layer, frontend layer, backend layer, Computer Vision layer, data layer, supporting services, and security layer. The user layer consists of the fleet manager, who accesses the application through a web browser, enters driver details and vehicle information, and views fatigue alerts and fleet safety reports, and the drivers, who register and log in and provide the real-time video feed. The frontend layer is developed using HTML, CSS, and JavaScript and provides driver login screens, admin control panels, live alerts, incident logs, and driver history. The backend layer uses Flask to handle incoming requests and application business logic and to connect the web interface with SQLite and the vision models. The Computer Vision

layer processes live video feeds from driver cameras using OpenCV and MediaPipe to detect drowsiness signs and triggers instant audio warnings and emergency calls. The data layer uses SQLite as the relational database to store driver records, vehicle details, and incident logs. The supporting services include audio processing using PyGame and Text-to-Speech, and optional external services such as email notifications, cloud backups, and emergency services. When a driver is being monitored, the video feed is sent to the Computer Vision engine, the drowsiness result is validated by the alert logic, the incident is stored in the database, and the alert and incident summary are displayed on the admin dashboard. This layered design ensures organized processing, maintainability, and consistent management of driver and incident data.

5.2.1 Client-Side Design

The client-side represents the user interface of the system. It is developed using HTML, CSS, and JavaScript to provide a simple, responsive, and user-friendly experience.

The client-side includes pages such as the home page, platform features, driver registration, sign-in, driver dashboard, trip incident history, and the admin control panel. HTML structures the pages, CSS provides the design and responsiveness, and JavaScript is used for client-side interactions and form validation.

Users can access the system through a web browser and perform actions such as registering, signing in, launching the camera monitor, viewing the audio alert diagnostics, and viewing the trip incident history. The driver dashboard summarizes the vehicle ID, driver ID, total incidents, AI guard status, and the incident history of the driver. The admin panel displays live alerts, incident logs, driver history, and reports.

5.2.2 Server-Side Design

The server-side is responsible for handling the application logic, processing user requests, and communicating with the database. It is developed using Flask with Python.

The server processes operations such as authentication and role control, driver and vehicle management, connection with the vision models, alert handling, incident logging, and report preparation. It communicates with the SQLite database to store and retrieve companies, users, vehicles, monitoring logs, incident reports, emergency responses, and driver history.

For monitoring, the Computer Vision engine analyzes the video feed, computes the eye aspect ratio and the duration of eye closure, and flags drowsiness when the eyes remain closed. The alert logic then triggers the voice warnings, beeps, and horn sounds, and the incident logging engine

stores the incident with its type, time, severity, and system response. When a user performs an action, the request is sent to the Flask server, validated, processed, and the appropriate response is returned to the client-side.

5.3 System Architecture

The system follows a layered architecture consisting of the Drivers and the Fleet Manager (Admin) interfaces, the Web Frontend, the Flask Backend, the SafeSteer AI Monitoring System (Core), the Database, and External Services.

The drivers register and log in through their devices and provide the real-time video feed. The Fleet Manager Dashboard, a Flask web application, allows the fleet manager to monitor and manage drivers, view incident reports, and view data analytics. The frontend uses HTML, CSS, and JavaScript to provide the login screens, admin control panels, and dashboards. Flask handles requests and application logic and connects the dashboard with the database and the vision models. The core SafeSteer AI monitoring system contains the Face Mapping Engine, the Alerting Logic, the Incident Logging Engine, and the Emergency Call Simulation, and it performs drowsiness detection using OpenCV and MediaPipe. The SQLite database stores the incidents logged by the system in a tamper-proof manner, and optional external services include email notifications, cloud backups, and emergency services.

The core control components of the system are the Face Mapping Engine, which maps the face of the driver from the live video feed and identifies eye-closure patterns; the Alerting Logic, which provides voice warnings, beeps, and horn sounds; the Incident Logging Engine, which records each incident in the database; and the Emergency Call Simulation, which simulates an emergency rescue call for critical incidents. Authentication and role-based permissions separate the actions of drivers and the fleet manager, and the overall data flow runs from the driver video feed through the core system to the database and the fleet manager dashboard.

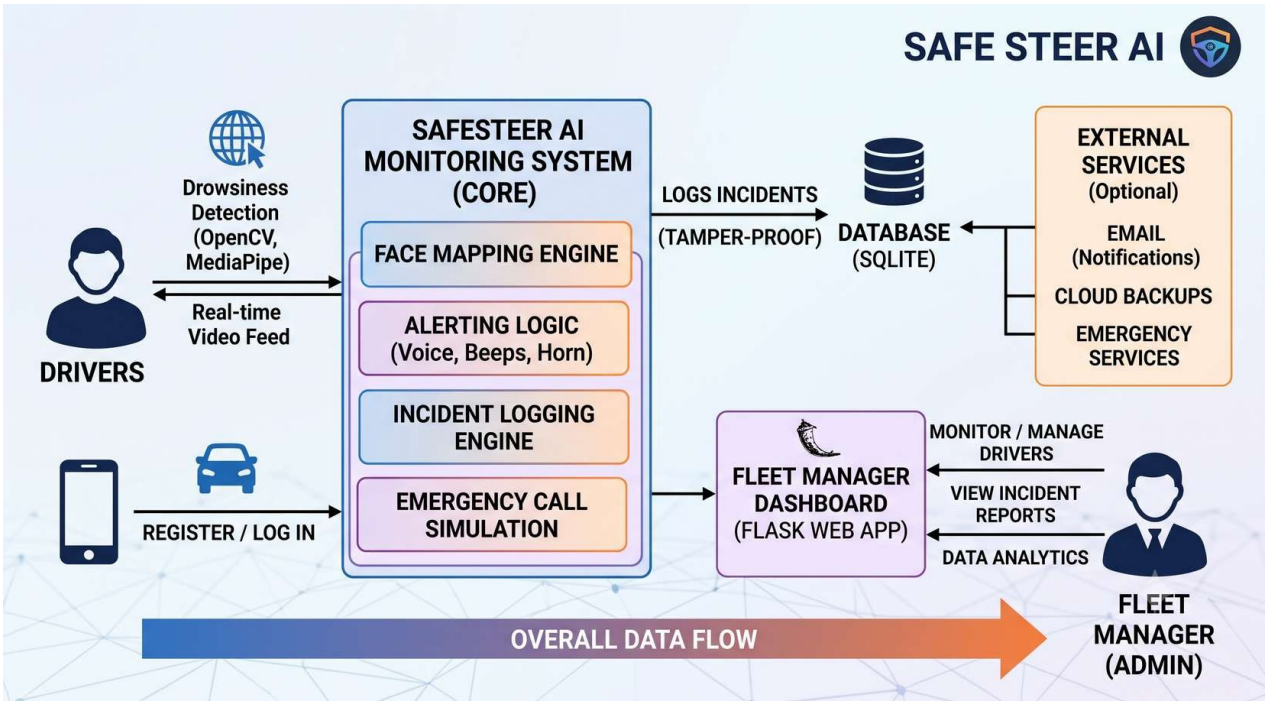


Figure 5.1: System Architecture

5.4 Use Case Diagram

The Use Case Diagram represents the interaction between the users and the SafeSteer AI Driver Monitoring and Fleet Safety System. It identifies the main users of the system and the activities that can be performed. In the diagram, there are two primary actors: **Driver (Employee)** and **Admin (Fleet Manager)**.

Actors:

1. **Driver (Employee):** The driver is the employee who drives the vehicle. The driver can log in and authenticate, view the personal dashboard, view the incident history of own records, and take part in real-time monitoring and drowsiness detection. The driver also receives alerts in the form of voice warnings, beeps, and horn sounds, and the emergency rescue simulation is triggered when a critical incident occurs.
2. **Admin (Fleet Manager):** The admin is the fleet manager who logs in and authenticates, monitors the drivers through real-time monitoring and drowsiness detection, receives and controls the alerts through system control (manage alerts), and manages users, vehicles, and reports.

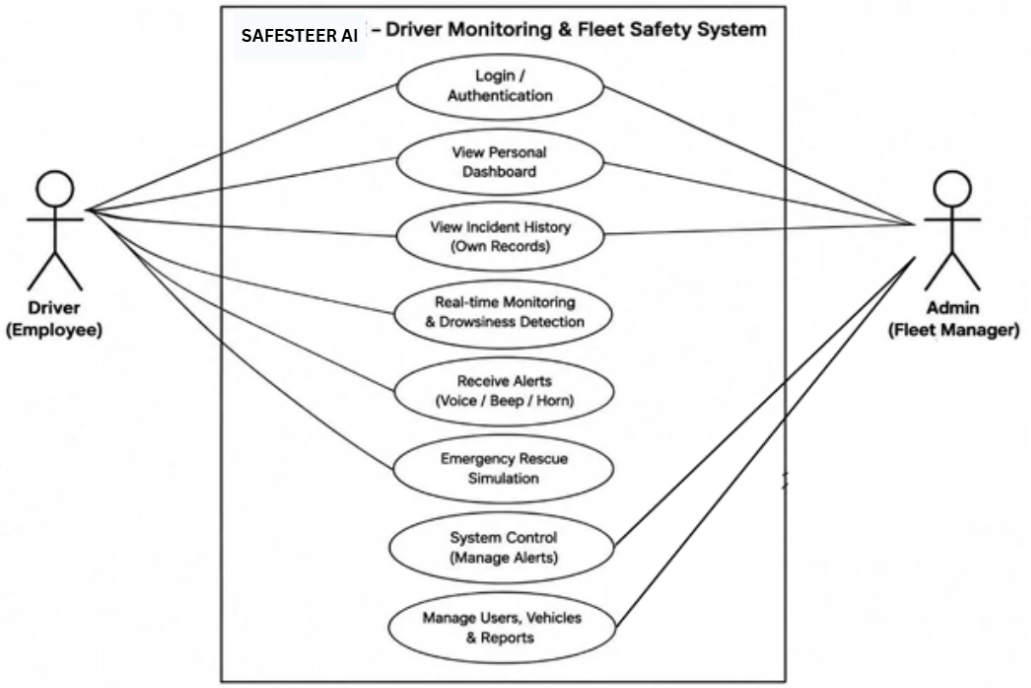


Figure 5.2: Use Case Diagram for SafeSteer AI-Driver Monitoring System

5.5 Data Flow Diagrams (DFD)

The Data Flow Diagram (DFD) illustrates how data flows within the system between the users, the application processes, and the databases. The driver sends registration and login details and the detection input to the system and receives the monitoring results in return. The admin sends login details and management requests and receives the user records and the checked results. The system stores and retrieves user details, monitoring information, and result data in the databases.

The Level 0 DFD represents the whole system as a single process, Safe Steer AI, interacting with the Admin and the User. The Admin logs in or registers, manages, and monitors the system, and the User views reports. The Level 1 DFD is shown separately for the two users of the system, the Admin and the Driver.

In the Admin Level 1 DFD, the admin logs in directly through the Login process (1.0). From the login, the admin can view and manage the users through the View Management process (1.1), which views, updates, and deletes the records stored in USER-DB, and can check the monitoring results through the actual check process (1.2), which sends the details to Result_DB and receives the result.

In the Driver Level 1 DFD, the driver registers through the Register process (1.0), which stores

the user details in USER-DB, and then logs in through the Login process (1.1) with a session login, where the user information is read from USER-DB. After login, the detection input is passed to the Enable AI-Monitoring process (1.2), whose data source is passed to the Model Deployment process (1.3), which stores the updated data in INFO-DB. The driver can also view the monitoring result through the View Result process (1.4), which reads the result information from RESULT-DB.

Overall, the DFD provides a clear understanding of how the different processes of the system interact and how data is processed, stored, and retrieved.

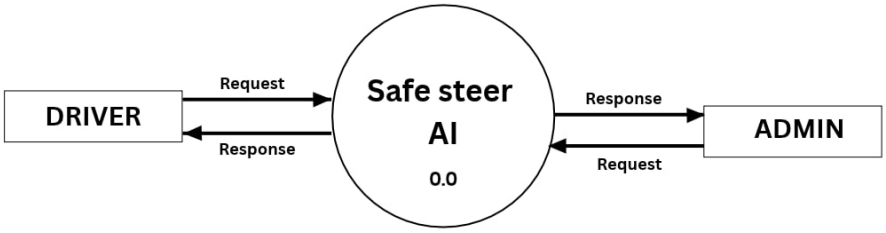


Figure 5.3: DFD Level 0

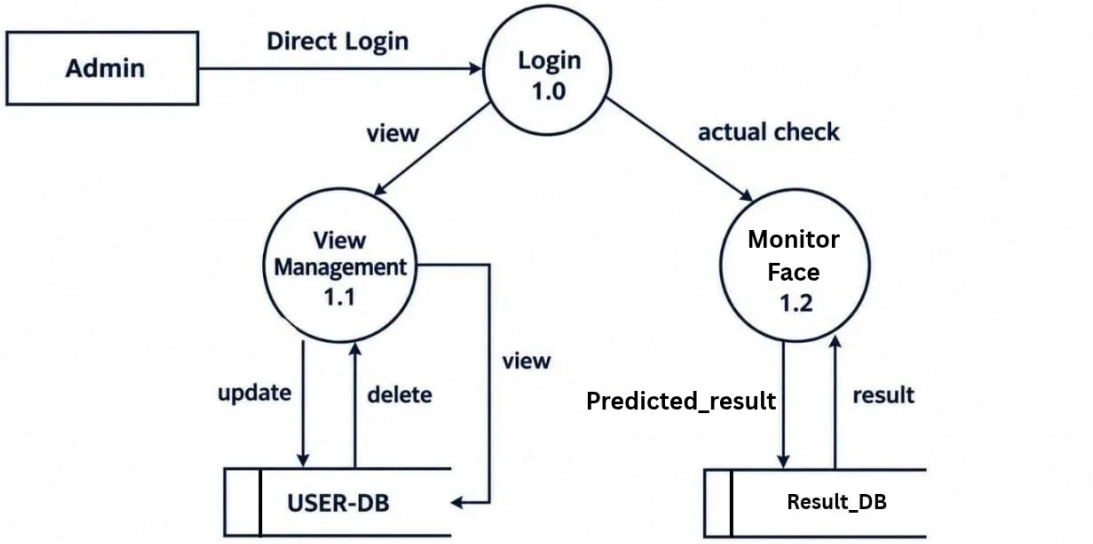


Figure 5.4: DFD Level 1 - Admin

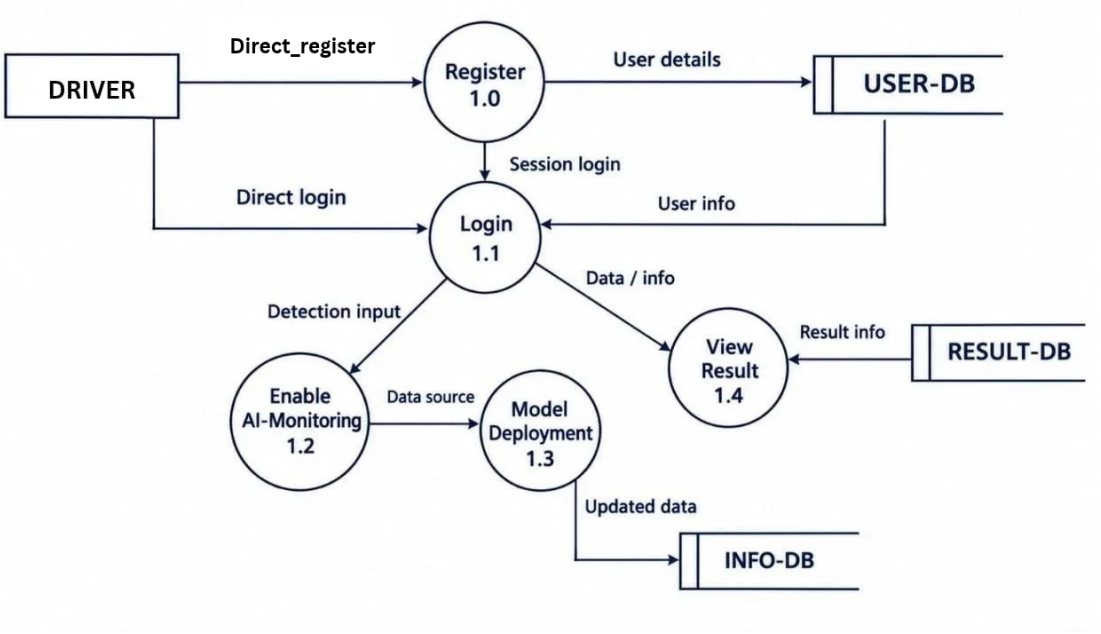


Figure 5.5: DFD Level 1 - Driver

5.6 Database Design

The system uses SQLite, a relational database, as its data store. It stores the data related to drivers, vehicles, distraction logs, admin details, and safety reports, and it maintains the relationships between drivers, vehicles, and incident records. The database stores the following information, and access to driver and incident data is kept secure and organized through validation and role-based access.

- Companies and the admin user of each company (companies)
- User and role information, including name, email, password hash, and role (users)
- Vehicle details and the assigned driver (vehicles)
- Monitoring logs with the event type, event time, drowsiness level, distraction type, and confidence (monitoring_logs)
- Incident reports with the incident type, time, system response, and severity (incident_reports)
- Emergency responses with the response type, time, and status (emergency_responses)
- Driver history linking the driver to the monitoring logs and incident reports viewed (driver_history)

5.7 Module Description

SafeSteer AI-Driver Monitoring System is designed as a role-based web application that enables the fleet manager to monitor driver safety and drivers to view their own records. It operates through

two primary modules **Management and Reporting Module and Monitoring and Detection Module** which work together to provide an organized driver safety platform.

5.7.1 Management and Reporting Module

This module covers the user interface, driver and vehicle management, data storage, and reporting functions.

1. **Frontend UI:** Provides a simple and responsive web portal for drivers and fleet managers and displays driver dashboards, vehicle details, safety alerts, and reports.
2. **Driver Management Module:** Adds, updates, views, and manages employee driver profiles and handles company login credentials and vehicle assignments.
3. **Database Module:** Uses SQLite to store driver profiles, vehicle data, and incident logs.
4. **Reports and Analytics Module:** Displays driver history, risky habits, and safety trends, and helps fleet managers review driver performance and reduce accidents.

5.7.2 Monitoring and Detection Module

This module is the Computer Vision and alerting component that analyzes the driver, triggers warnings, and records incidents.

1. **Monitoring and Alert Module:** Analyzes live driver video feeds in real time and triggers voice warnings, beeps, and horn sounds during drowsiness.
2. **Drowsiness Detection Module:** Processes face landmarks using OpenCV and MediaPipe and detects eye closures to spot fatigue signs.
3. **Incident Logging Module:** Captures the distraction type, time, and system alerts and stores tamper-proof logs in the database for fleet review.
4. **Emergency Call Simulation:** Simulates an emergency rescue call and records the response when a critical incident is detected.

CHAPTER 6

REPORT OF PROJECT IMPLEMENTATION

6.1 Implementation Plan

6.1.1 Setup and Environment Preparation

- Set up the development environment using Python, Flask, and Visual Studio Code on Windows, Linux, or macOS.
- Install the required Python libraries for Flask, OpenCV, MediaPipe, PyGame, and Text-to-Speech.
- Configure the SQLite database to store company, user, vehicle, monitoring log, incident report, emergency response, and driver history information.
- Create the necessary database tables for companies, users, vehicles, monitoring_logs, incident_reports, emergency_responses, and driver_history.
- Organize the project into modules for the frontend, driver management, monitoring and alerts, drowsiness detection, incident logging, database, and reports and analytics.
- Connect a web camera for capturing the live driver video feed.

6.1.2 Backend Implementation

- Develop the backend using Flask to handle requests, routing, and application workflows.
- Implement driver registration, login, and logout with separate permissions for drivers and the fleet manager.
- Implement the Computer Vision module that captures the video feed and tracks the face using OpenCV and MediaPipe.
- Implement drowsiness detection using eye closure patterns, the eye aspect ratio, and the time duration.
- Implement the alert logic for voice warnings, beeps, and horn sounds using PyGame and Text-to-Speech.

- Implement the incident logging engine to store the distraction type, time, and severity in the database.
- Implement the emergency call simulation for critical incidents.
- Implement the driver and vehicle management operations to add, update, delete, search, and retrieve records.
- Implement the data for driver history, incident trends, and risk levels.

6.1.3 Frontend Implementation

- Develop the user interface using HTML, CSS, and JavaScript.
- Design pages for the home page, platform features, driver registration, sign-in, driver dashboard, and trip incident history.
- Design the control panel and dashboard for the fleet manager.
- Create forms with basic input validation and user feedback to prevent invalid data entry.
- Display live alerts, incident logs, and driver history on the dashboards.
- Display the audio alert diagnostics, including the voice warnings, system boot, drowsy pulse, distraction scan, safe recovery, and emergency klaxon.
- Display the driver ID, vehicle ID, total incidents, and AI guard status on the driver dashboard.
- Design the interface to be responsive and suitable for mobile phone, tablet, and desktop browser sizes.

6.1.4 Security and Data Handling

- Implement authentication to protect driver accounts and the admin control panel.
- Implement role-based access to separate the actions of drivers and the fleet manager.
- Allow drivers to view their own incidents and profile in read-only form, with no modification of records.
- Validate login credentials before granting access to protected pages.
- Validate form inputs before processing.
- Store the incident logs in the database as tamper-proof records for safety audits.
- Store driver, vehicle, and incident information in the relational database.

- Ensure that protected operations are accessible only to authenticated users with the required role.

6.1.5 Project Schedule

The project was planned over a period of 16 weeks from July 2026 to October 2026, with the following tasks and durations.

- Literature Review: 2 weeks
- Requirement Analysis: 3 weeks
- System Design and Architecture: 3 weeks
- CV Model Setup (OpenCV/MediaPipe): 3 weeks
- Drowsiness Detection and Alert Logic: 4 weeks
- Frontend and Backend Development: 4 weeks
- Database Integration (SQLite): 3 weeks
- System Testing and Evaluation: 3 weeks
- Reports and Documentation: 4 weeks
- Final Presentation: 1 week

The key milestones of the project were Requirement Complete (end of July 2026), Model Logic Ready (mid September 2026), Full System Integration (end of September 2026), and Final Project Review (end of October 2026).

6.2 Testing and Various Types of Testing Used

Testing verifies that the software functions correctly according to its requirements, helping identify errors and ensure reliable interaction between the frontend, backend, database, and Computer Vision modules. In the SafeSteer AI-Driver Monitoring System, testing covers driver registration and login, driver and vehicle management, drowsiness detection, audio alerts, incident logging, emergency call simulation, database accuracy, input validation, and access control.

6.2.1 Unit Testing

Unit testing checks individual functionalities independently to verify that each performs its intended task. In this system, individual features such as registration, login, driver creation, eye aspect ratio calculation, drowsiness duration check, alert triggering, and incident log creation are tested sepa-

rately. This helps identify errors in individual components before testing the complete application.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	Driver Registration	A new driver account is created with valid details.	A new driver account was created and saved in the database.	Pass
2	User Login	A registered user logs in using valid credentials and is directed to the interface for the role.	The user logged in successfully and was directed to the dashboard for the role.	Pass
3	Invalid Login	Incorrect login credentials are rejected with an error message.	Incorrect credentials were rejected and an error message was displayed.	Pass
4	Add Driver and Vehicle	A driver profile and the assigned vehicle are saved in the database.	The driver profile and the assigned vehicle were saved and displayed in the records.	Pass
5	Eye Aspect Ratio Calculation	The eye aspect ratio is calculated from the face landmarks of the video frame.	The eye aspect ratio was calculated correctly from the face landmarks.	Pass
6	Drowsiness Duration Check	Drowsiness is flagged when the eyes remain closed for longer than the set duration.	Drowsiness was flagged when the eyes stayed closed beyond the set duration.	Pass
7	Alert Triggering	A voice warning, beep, or horn sound is triggered when drowsiness is detected.	The voice warning, beep, and horn sound were triggered on detection.	Pass
8	Incident Log Creation	An incident is stored with its type, time, and severity.	The incident was stored with its type, time, and severity.	Pass

Table 6.1: Unit Test Cases and Results

6.2.2 Integration Testing

Integration testing verifies that different components of the system work together correctly. It checks the interaction between the frontend, Flask backend, SQLite database, Computer Vision modules, and audio alert services. The testing focuses on whether user actions are processed correctly, database changes are reflected in the interface, and detection and alert results are returned and displayed as expected.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	Registration and Database Integration	Valid driver details are stored in the database.	The driver details were stored correctly in the database.	Pass
2	Login Integration	The backend verifies credentials and creates the appropriate session and role.	The credentials were verified and the session and role were created correctly.	Pass
3	Video Feed and Detection Integration	The live camera feed is processed by the Computer Vision module and the driver state is detected.	The live feed was processed and the driver state was detected.	Pass
4	Alert Integration	A detected drowsiness event triggers the audio alerts through the audio processing module.	The detected drowsiness event triggered the audio alerts.	Pass
5	Incident Logging Integration	A detected incident is stored in the database and appears in the incident logs of the dashboard.	The incident was stored and appeared in the incident logs of the dashboard.	Pass
6	Emergency Call Integration	A critical incident triggers the emergency call simulation and the response is recorded.	The emergency call simulation was triggered and the response was recorded.	Pass
7	Driver Dashboard Integration	The driver details and incident history are displayed correctly on the driver dashboard.	The driver details and incident history were displayed correctly.	Pass
8	Analytics Integration	Stored incident data are retrieved correctly to display driver history and incident trends.	The stored incident data were retrieved and the driver history and trends were displayed.	Pass

Table 6.2: Integration Test Cases and Results

6.2.3 System Testing

System testing evaluates the complete application to verify that its major features operate together as expected. It includes testing user access, driver management, live monitoring, alerts, incident logging, administration, and database operations. It also checks input validation and access restrictions

to identify problems that may affect the overall reliability and usability of the application.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	Overall System Testing	The major features of the application operate correctly.	The major features operated correctly together.	Pass
2	Authentication Testing	Valid credentials are accepted and invalid credentials are rejected.	Valid credentials were accepted and invalid credentials were rejected.	Pass
3	Driver Management Testing	Driver records can be added, updated, searched, and deleted by the admin.	The admin could add, update, search, and delete driver records.	Pass
4	Monitoring Testing	Live driver video is analyzed and drowsiness is detected from eye closures.	The live video was analyzed and drowsiness was detected from eye closures.	Pass
5	Alert and Logging Testing	Alerts are triggered and the incidents are logged with the date, time, and severity.	The alerts were triggered and the incidents were logged with date, time, and severity.	Pass
6	Operator and Admin Testing	Drivers and the admin can perform only their permitted operations.	Drivers and the admin could perform only their permitted operations.	Pass
7	Input Validation Testing	Invalid or incomplete input is rejected appropriately.	Invalid or incomplete input was rejected with a message.	Pass
8	Access Control Testing	Users cannot access pages or records that are not permitted for their roles.	Access to pages and records not permitted for the role was denied.	Pass

Table 6.3: System Test Cases and Results

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Advantages

- **Integrated Platform:** The system combines live driver monitoring, drowsiness detection, audio alerts, emergency simulation, incident logging, and reporting in one application.
- **Real-Time Drowsiness Detection:** Live video feeds are analyzed using Computer Vision to detect drowsiness and distraction right away, which helps prevent accidents caused by driver sleepiness.
- **Instant Multi-Stage Audio Alerts:** The driver is warned through voice warnings, beeps, and horn sounds, together with emergency call warnings.
- **Tamper-Proof Incident Logs:** Every incident is stored with its type, time, and severity in the database for fleet review and safety audits.
- **Fleet Manager Control:** The fleet manager can manage drivers and vehicles and view live alerts and incident summaries directly on the admin dashboard.
- **Role-Based Workflows:** Separate interfaces and permissions for drivers and the fleet manager keep responsibilities clear and secure.
- **Dashboards and Analytics:** Driver history, risky habits, incident trends, and risk levels support better fleet safety decisions.
- **Simple and Practical Design:** The system uses Flask, SQLite, OpenCV, and MediaPipe and runs on a computer with a web camera, which makes it simple and practical for an organization.
- **Responsive Interface:** The web interface works across mobile phone, tablet, and desktop browsers.
- **Scalable Design:** Separate modules and relational storage support the addition of more drivers and vehicle logs in the future.

7.2 Limitations

- **Dependency on Camera and Lighting:** The detection depends on a clear view of the face of the driver, and performance can be affected by poor lighting, occlusion, and sunglasses.
- **Emergency Call Is Simulated:** The emergency call is only simulated and the system does not connect to real emergency services.
- **Limited Detection Indicators:** Drowsiness is detected mainly from eye closure, the eye aspect ratio, and the time duration, and advanced multimodal detection is planned as a future extension.
- **Limited Evaluation:** The application has been demonstrated with a small number of drivers and vehicles and has not been evaluated in a large real-world fleet deployment.
- **Database Scalability:** SQLite is suitable for a small organization, and a larger deployment would need a more scalable database server.
- **Basic Notification Mechanism:** Alerts and incident summaries are displayed within the application, and external notification services such as email are optional and not fully implemented.
- **Security Enhancements Needed:** Although authentication and role-based access are used, stronger measures such as encryption and audit logging would be needed for a production deployment.

7.3 Screenshots

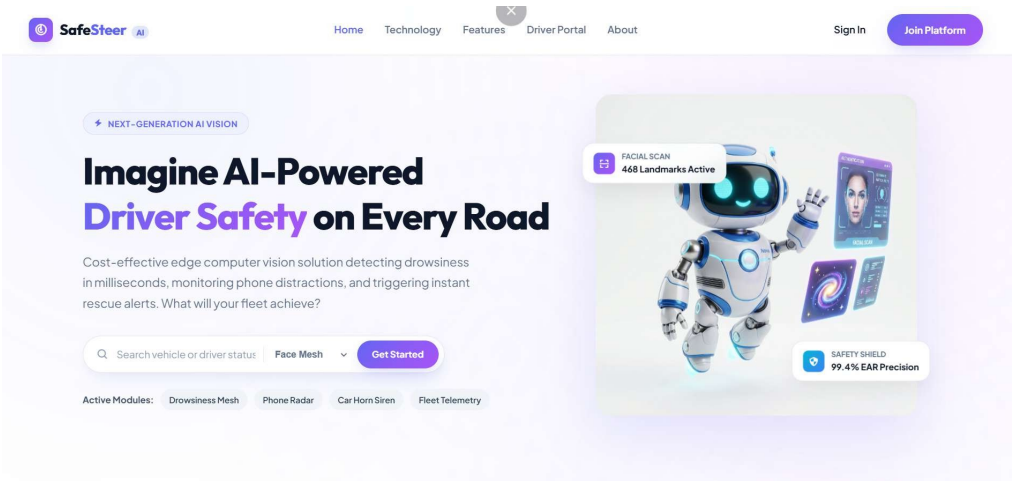


Figure 7.1: Home Page

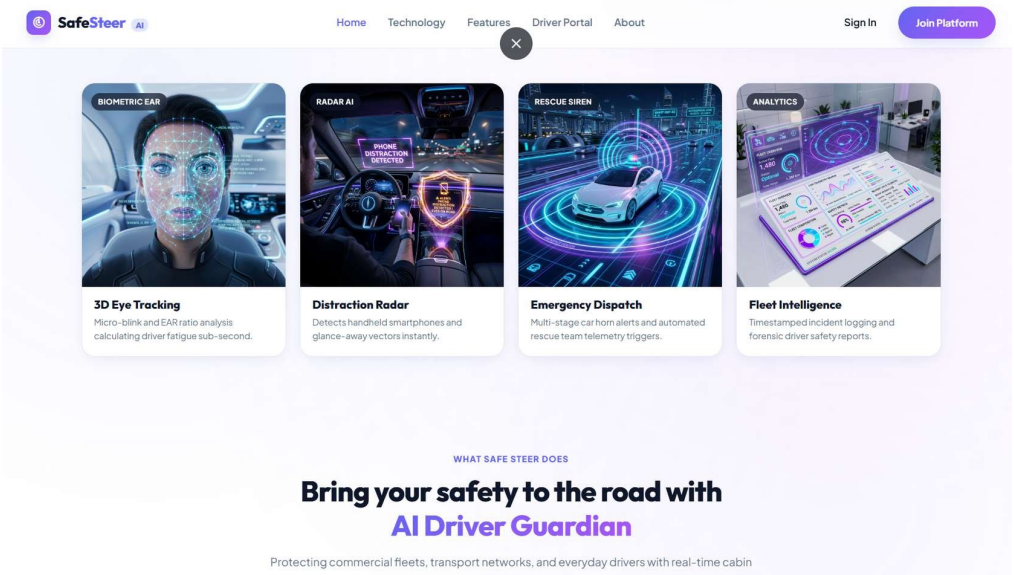


Figure 7.2: Platform Features

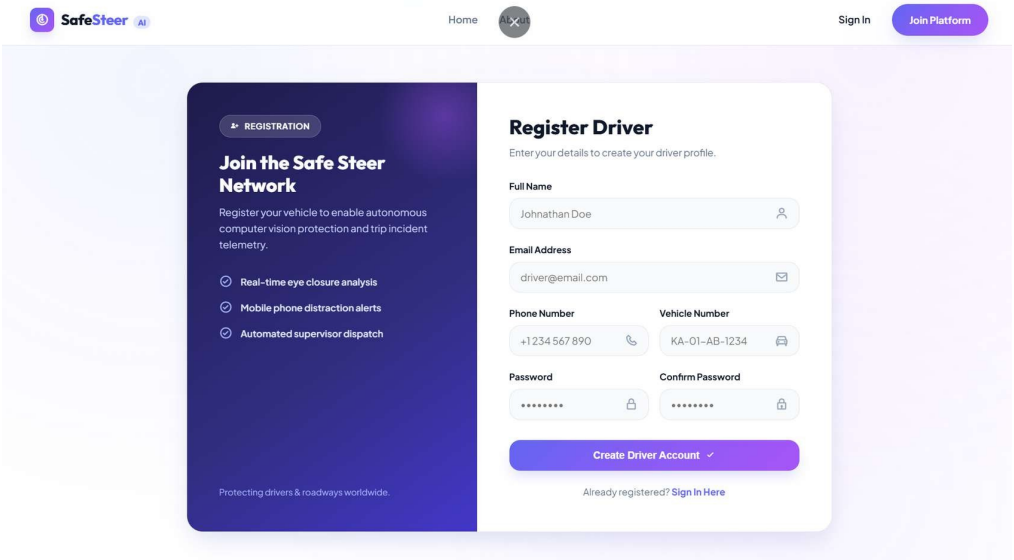


Figure 7.3: Driver Registration

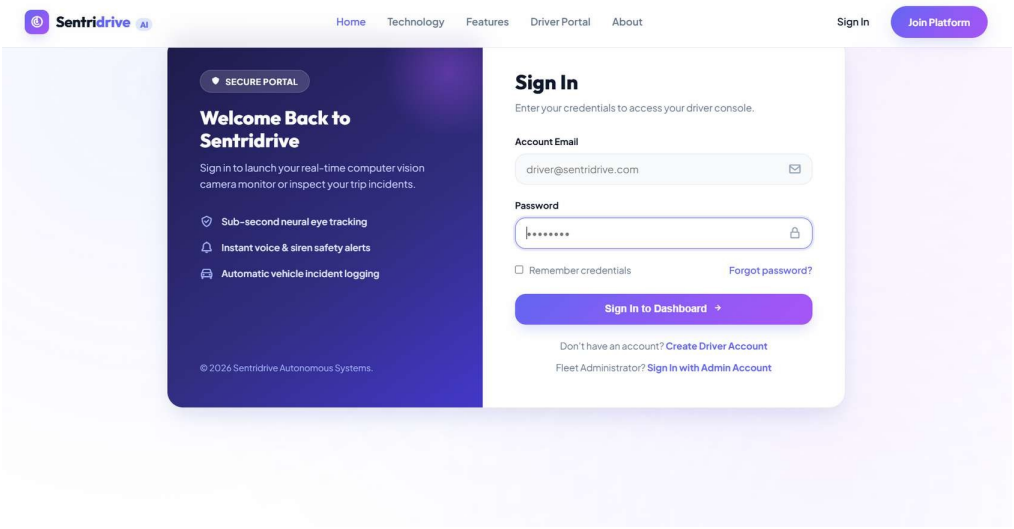


Figure 7.4: Sign In

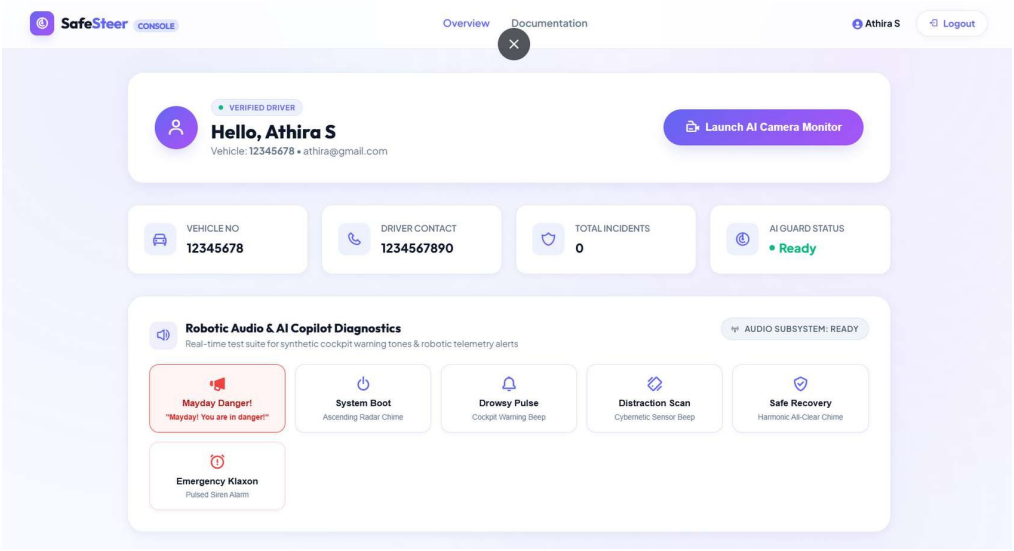


Figure 7.5: Driver Dashboard

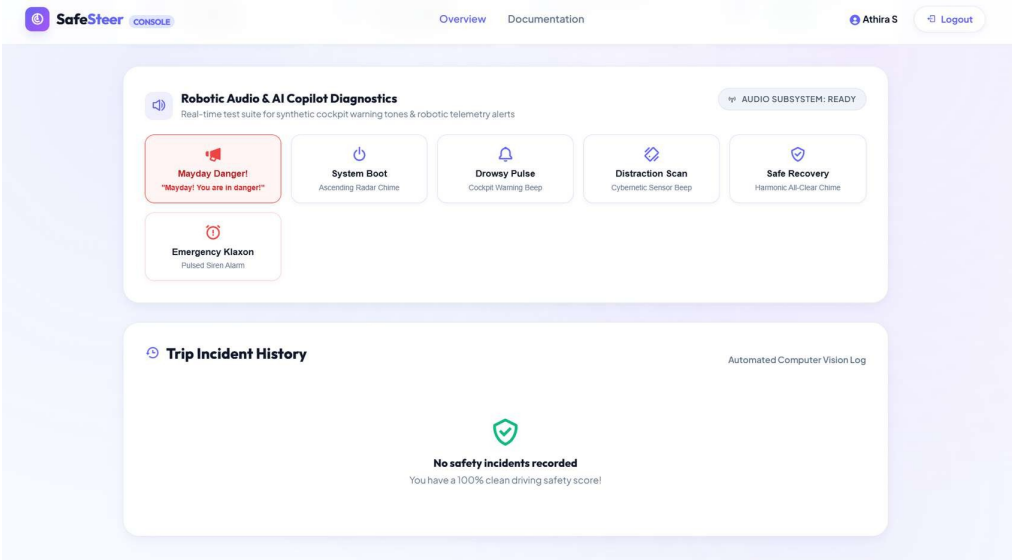


Figure 7.6: Trip Incident History

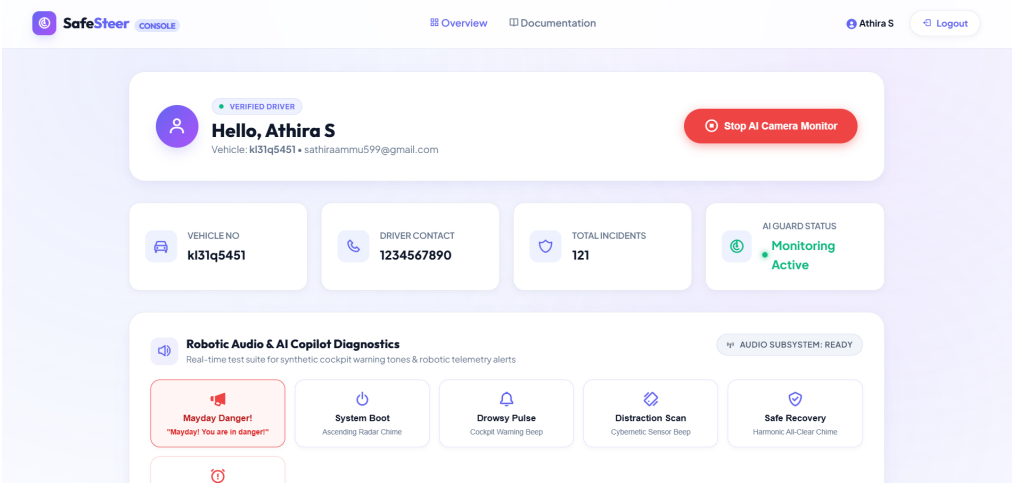


Figure 7.7: Driver Dashboard with Active AI Monitoring

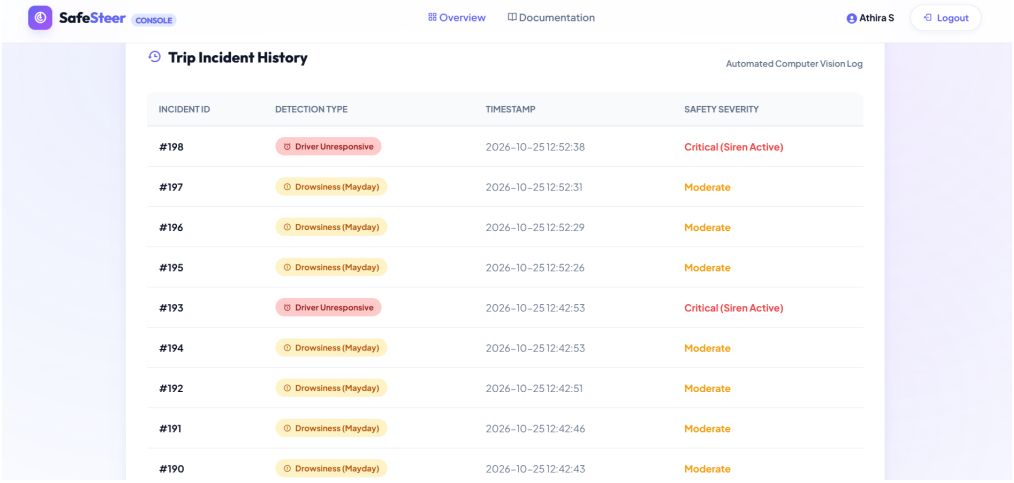


Figure 7.8: Trip Incident History with Detected Incidents

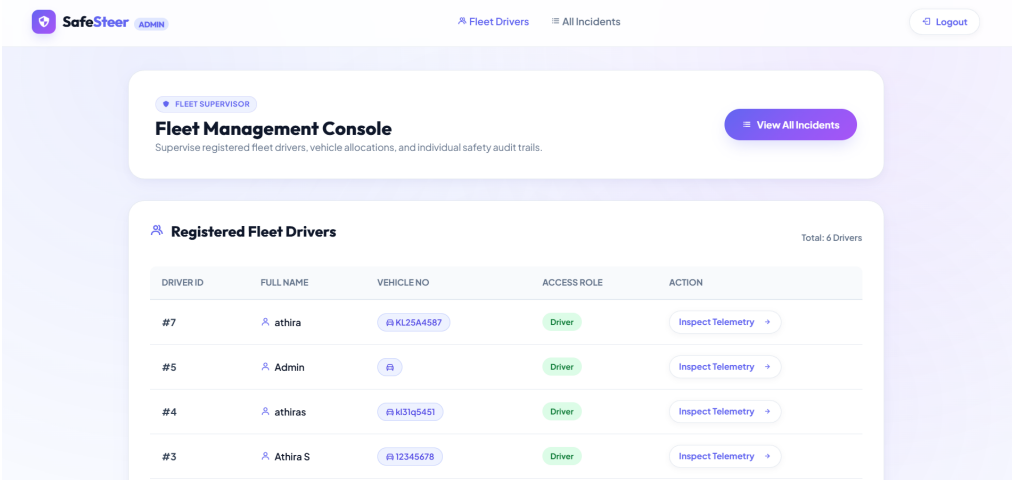


Figure 7.9: Admin Fleet Management Console

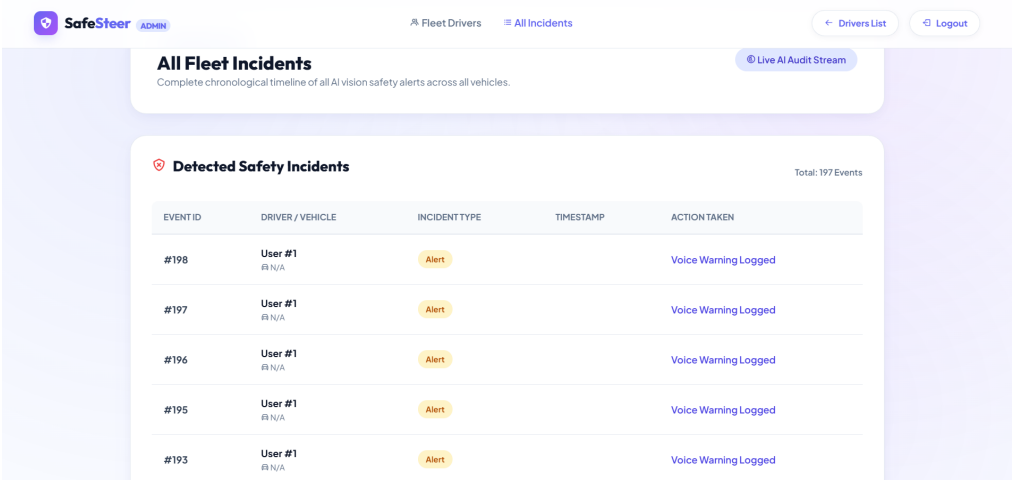


Figure 7.10: Admin All Fleet Incidents

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The development of the SafeSteer AI-Driver Monitoring System addresses a practical safety problem in fleet management. The system helps fleet managers monitor and ensure driver safety on the road. By using live camera feeds and Computer Vision for real-time fatigue detection, together with instant audio alerts and an emergency call simulation, the system warns drivers before drowsiness leads to an accident.

The system helps maintain tamper-proof incident logs and supports better fleet control and organizational safety decisions through driver history, incident trends, and risk levels. Role-based workflows separate the responsibilities of drivers and the fleet manager. Overall, the project provides a simple, practical, and secure organizational AI system that helps prevent road accidents and monitor driver safety efficiently.

8.2 Future Scope

1. **Advanced Deep Learning Models:** Using transformer-based or attention-enhanced models to improve detection accuracy under varying lighting, occlusion, and sunglasses.
2. **Multimodal Detection:** Combining eye closure with other indicators such as yawning, head pose, and facial landmarks to reduce dependence on a single indicator.
3. **Real Emergency Service Integration:** Replacing the emergency call simulation with integration to real emergency services and sending notifications to the fleet manager.
4. **Mobile Application and Notifications:** Developing a mobile application and adding email, SMS, or push notifications for alerts and incident reports.
5. **Cloud Backup and Scalable Database:** Adding cloud backups and a more scalable database for a larger number of drivers, vehicles, and incident logs.

REFERENCES

[1] A. A. Abd El-Nabi *et al.*, "Dynamic driver drowsiness detection with attention-enhanced CNNs for real-time monitoring and road safety applications," *Scientific Reports*, vol. 16, 2026.

[2] O. F. Hassan, A. F. Ibrahim, A. Gomaa, M. A. Makhlouf, and B. Hafiz, "Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach," *Scientific Reports*, vol. 15, no. 1, Art. no. 17493, May 2025, doi: 10.1038/s41598-025-02111-x.

[3] A. Bhanja, D. Parhi, D. Gajendra, K. Sinha, and A. K. Sahoo, "Driver drowsiness shield (DDSH): a real-time driver drowsiness detection system," *ROBOMECH Journal*, vol. 12, no. 1, Art. no. 18, May 2025, doi: 10.1186/s40648-025-00307-4.

[4] J. Fonseca and J. C. Ferreira, "Drowsiness detection in drivers: a systematic review of deep learning-based models," *Applied Sciences*, vol. 15, no. 16, Art. no. 9018, 2025.

[5] A. G. Sawant, S. S. Kamble, R. S. Kanade, R. N. Kanugo, T. A. Kapse, and K. A. Bhapse, "A real-time driver drowsiness detection system using MediaPipe and eye aspect ratio," arXiv:2511.13618, Nov. 2025.

[6] T. Soukupová and J. Čech, "Real-time eye blink detection using facial landmarks," in *Proc. 21st Computer Vision Winter Workshop*, Rimske Toplice, Slovenia, Feb. 2016.

[7] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," arXiv:1907.06724, Jul. 2019.

[8] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, Jun. 2019.